



Day 3: GPU Computing in Python

BASICS OF PERFORMANCE ANALYSIS AND PROFILING WITH NSIGHT COMPUTE PROFILING TOOL (NCU)

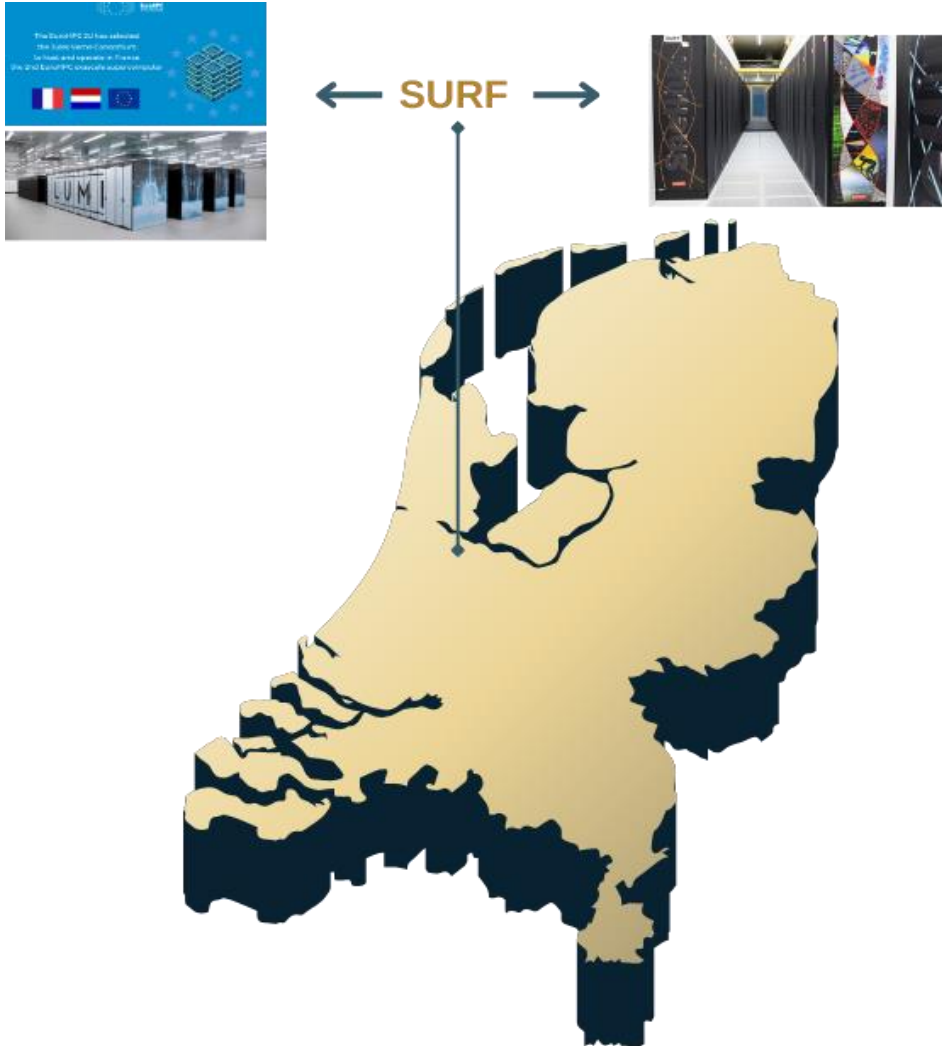
Clícia Pinto and Mohsen Safari
HPC Advisors, SURF

Jun 12th

Amsterdam Science Park



Who we are



SURF Expertise Pool:

- High-Performance Computing
- High-Performance Machine Learning
- High-performance visualization
- Quantum Computing
- Data preservation
- Data management
- Cloud for Research
- System administration
- Trust and Identity
- Software Management

Outline

- Profiling Tool - What it is and does
- Nsight Compute Profiling Tool (ncu)
- Profiling Applications on Snellius

Resources

- **Slides and source code of the examples can be found at:**
 - <https://github.com/sara-nl/Parallel-and-GPU-programming-in-Python/tree/main/Day2>
- **Programming environment (jupyter):**
 - <https://ondemand.snellius.surf.nl/>
- **To reset your password and Accept the User Agreement:**
 - <https://portal.cua.surf.nl>
- **To have more information on Snellius partitions and hardware:**
 - <https://servicedesk.surf.nl/wiki/spaces/WIKI/pages/30660209/Snellius+partitions+and+accounting>

Profiling Tool

What it is and does

- Characterizes hardware **performance metrics**
- Detects **performance issues**
- Supports **analysis** workflows
- Enables **baseline comparisons** across GPU architectures for cross-platform development
- Shows how performance relates to **underlying hardware concepts**
- Helps users identify **potential bottlenecks** and **underutilization**
- Correlates **efficiency** and **performance** metrics down to individual **lines of code**

Profiling Tool

What it is and does

Performance:

- **How fast** a task is completed by the computer
 - Execution time
 - Throughput
 - Bandwidth
 - FLOPS

Efficiency:

- **How well** the computational resources are used to achieve a given performance
 - Resource utilization
 - % of peak performance

Profiling Tool

What it is and does

- Performance Analysis
- Characterizes your workflow in
 - **Memory-bound**
 - Memory throughput is high
 - **Compute-bound**
 - Compute throughput is high

Profiling Tool

What it is and does

GPU Performance:

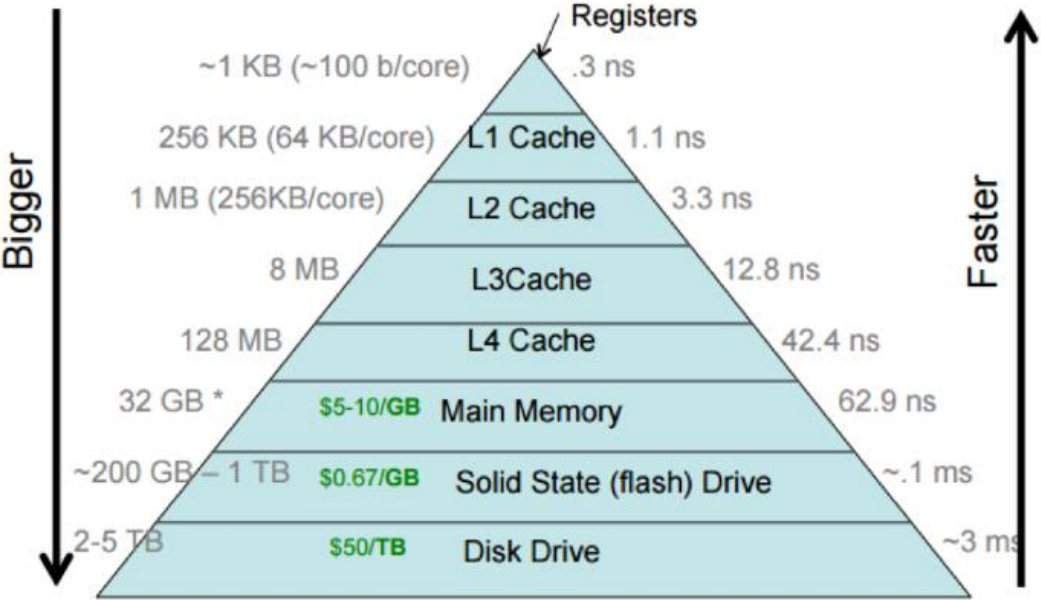
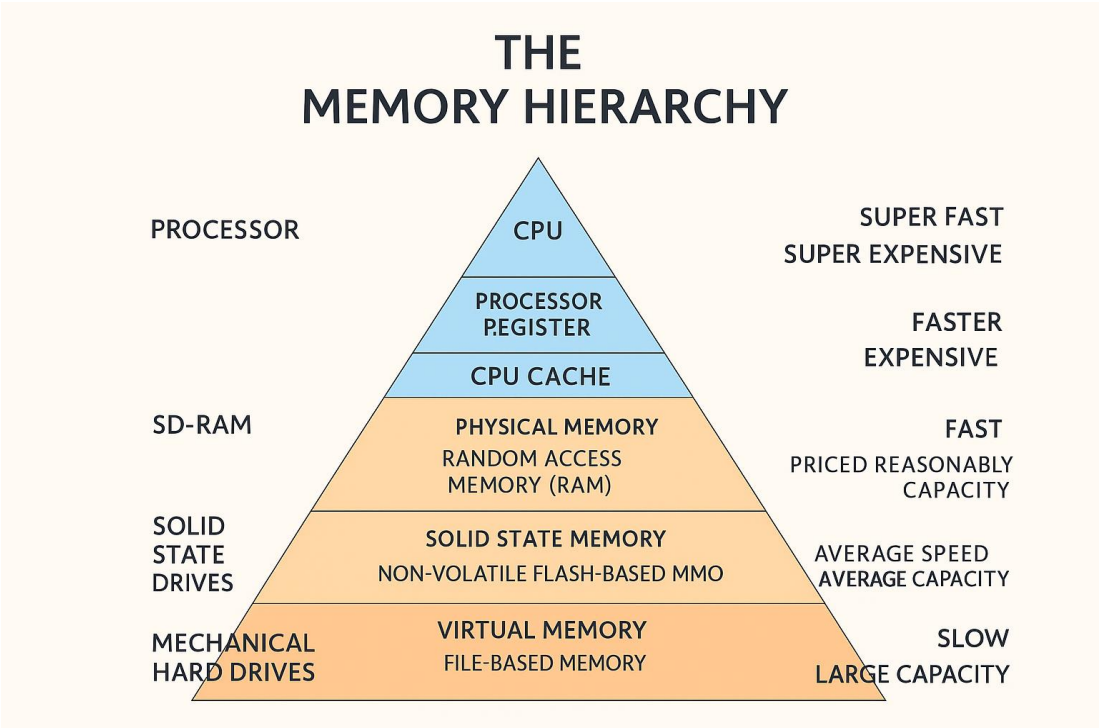
- **Unoptimized GPU programs are expensive** in terms of time, power and money
- Optimized GPU programs need to take advantage of several **execution units**
- High **trade-off between** development **time** and optimization **effort**

Think about:

- What is the purpose of using GPUs, and how is this code expected to evolve over time?
- What do I need in terms of performance? Fast execution times? Better throughput? Better occupancy of the GPU node?

Profiling Tool

Memory Hierarchy



Profiling Tool

Which tasks are best candidates for GPU?

✓ **Best** suited for GPU acceleration:

- High data parallelism
- Tasks that perform the same operations on many independent data elements.
- High computational intensity and low memory intensity
- Large amounts of computation per byte transferred.
- Examples include:
- Dense linear algebra
- Matrix–matrix multiplication
- Simulation kernels
- FFTs
- Monte Carlo methods
- Convolutions
- Physics simulations

✗ **Less** suitable for GPUs:

- Memory-heavy applications
- Workloads dominated by data movement rather than computation
- Frequent, small I/O operations
- Especially between host and device or network I/O.
- Irregular memory access patterns
- Unpredictable or scattered data access reduces GPU efficiency.
- Significant branching or divergence
- Heavy use of if–else, switch, or other divergent control flow.

Nsight Compute Profiling Tool (ncu)

What it is

- Interactive **kernel profiler** for CUDA applications
- Identifies **performance** and **efficiency** bottlenecks
- Analyses the device **memory usage**
- Translates metric values into **actionable** information
- It provides:
 - Detailed **performance metrics**
 - **API debugging**
- Can be used through **user interface (UI)** and **command line (CLI)** tool
 - (This course will cover the command line tool part)
- Differs from Nsight System (Nsys) profiler that provides **system-wide** profiling
- Ncu offers **kernel-based** profiling

Nsight Compute Profiling Tool (ncu)

Where to find

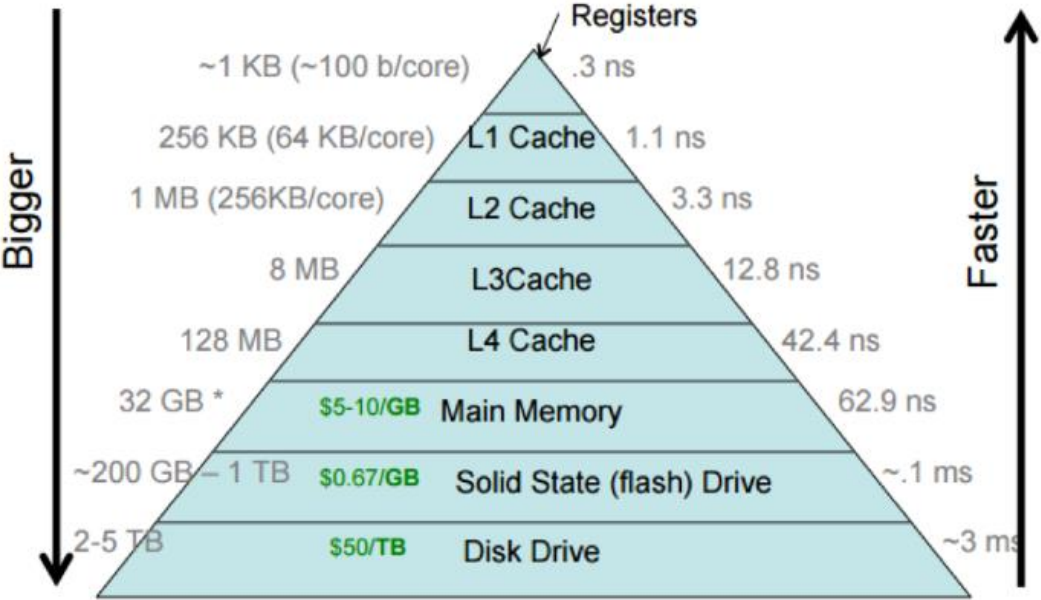
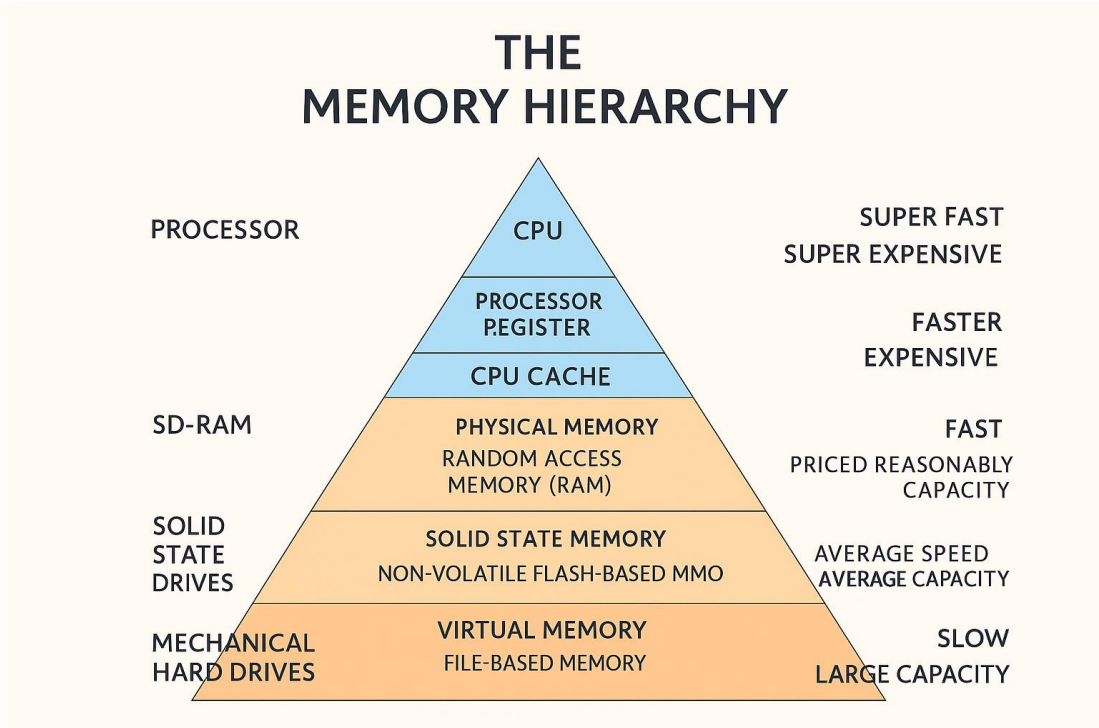
- Is available under the regular CUDA Toolkit installation
- On Snellius:

module load 2025

module load CUDA/12.8.0

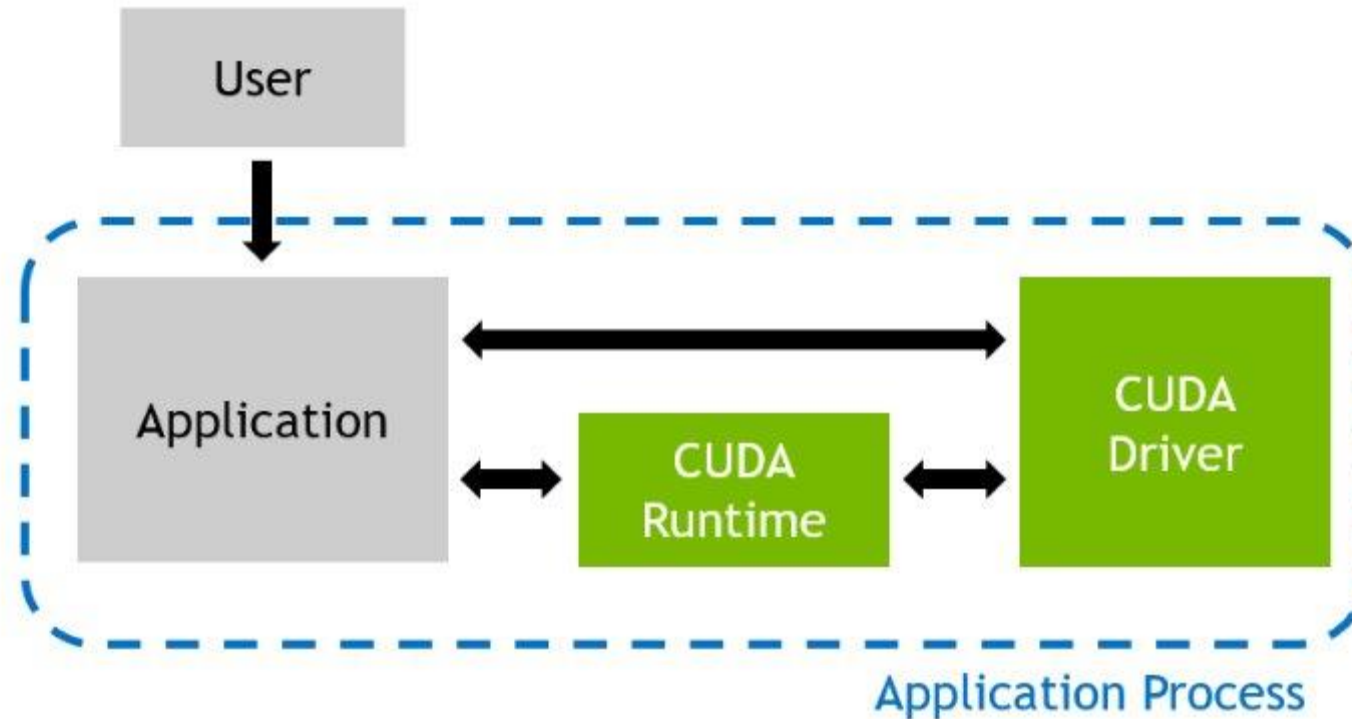
Profiling Tool

Memory Hierarchy



Nsight Compute Profiling Tool (ncu)

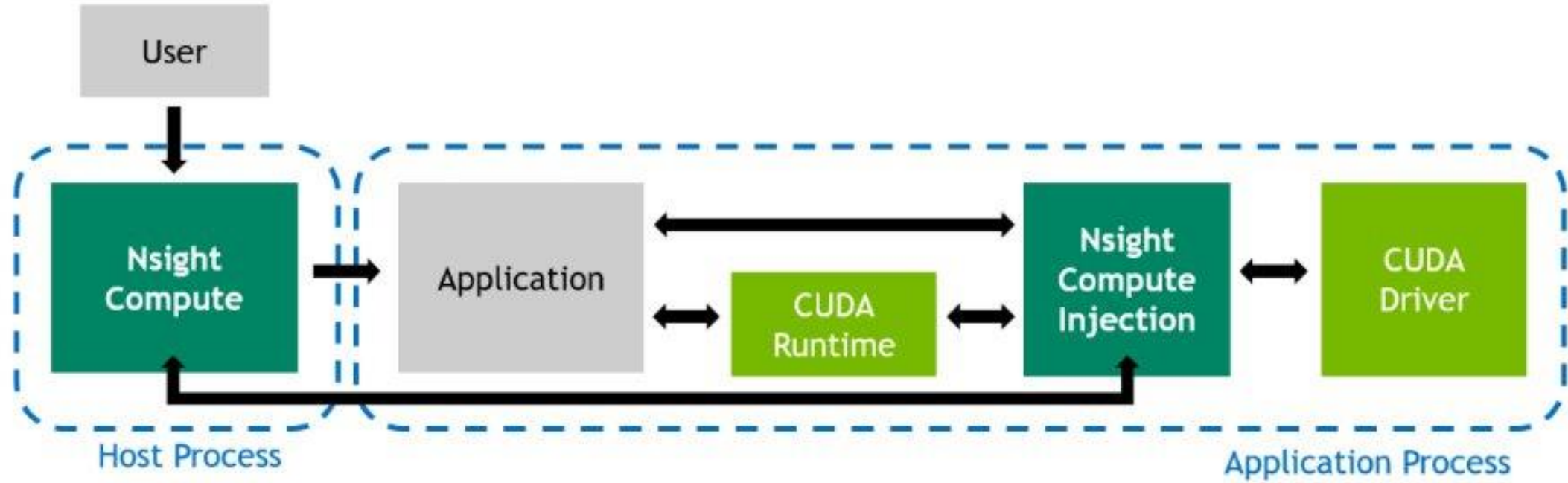
Profiling Applications



Regular Application Execution

Nsight Compute Profiling Tool (ncu)

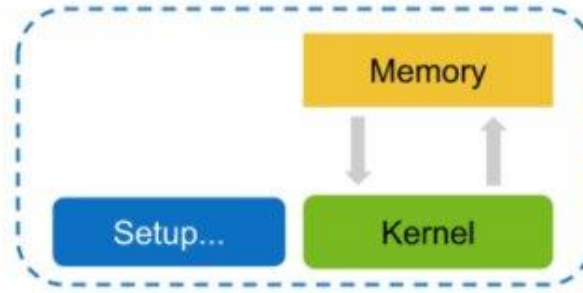
Profiling Applications



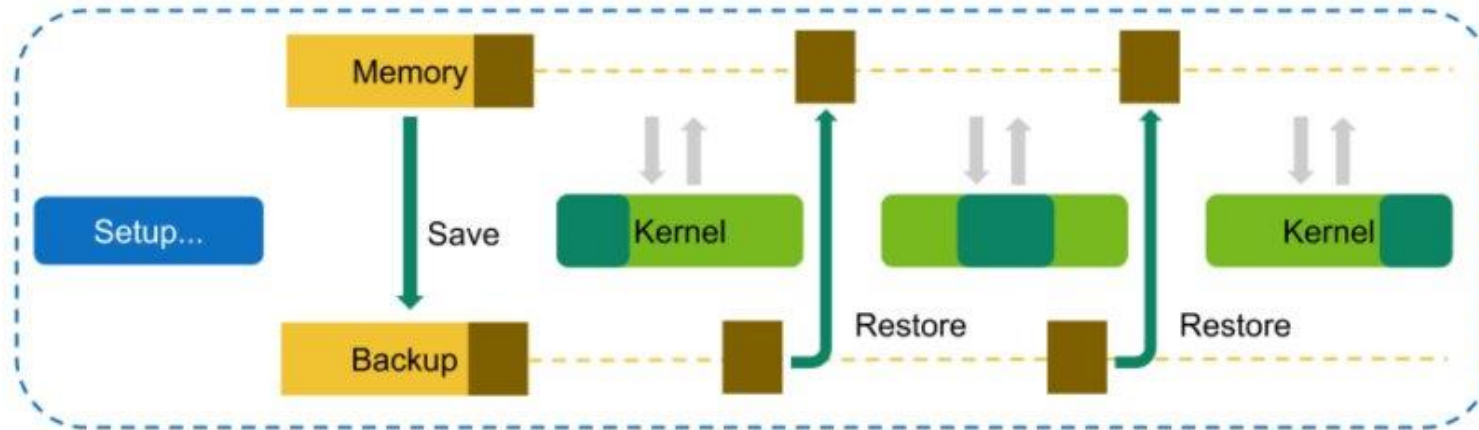
Profiled Application Execution

Nsight Compute Profiling Tool (ncu)

Metric Collection: Kernel Replay



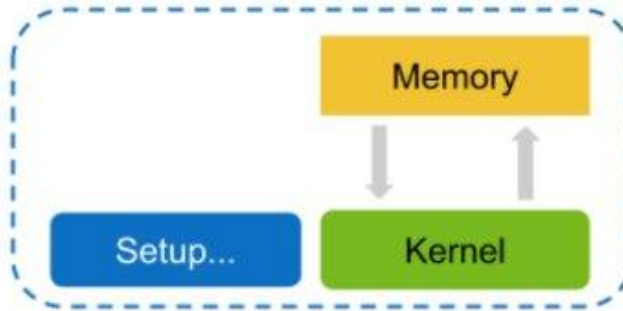
Regular Application Execution



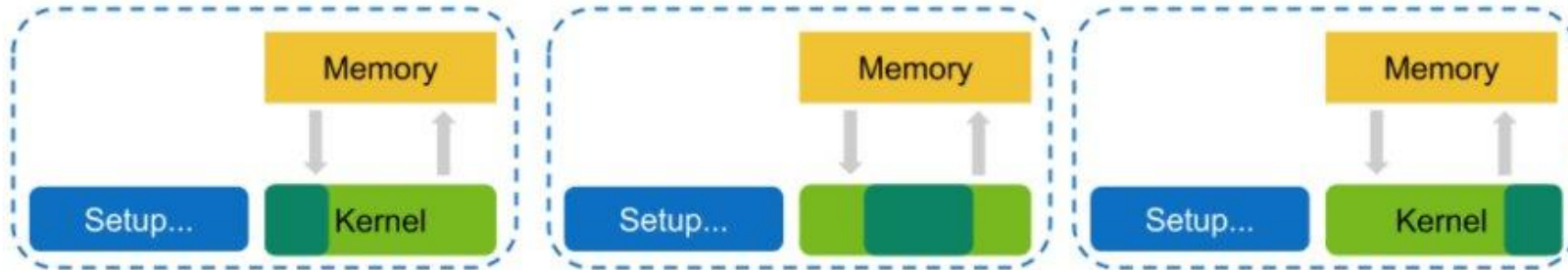
Execution with Kernel Replay. All memory is saved, and memory written by the kernel is restored in-between replay passes.

Nsight Compute Profiling Tool (ncu)

Metric Collection: Application Replay



Regular Application Execution



Execution with Application Replay. No memory is saved or restored, but the cost of running the application itself is duplicated.

Nsight Compute Profiling Tool (ncu)

Metric Collection: Sets and Sections

- Commonly used subset of metrics are grouped into pre-defined sets or sections
- Each set and section added adds also an overhead to the whole collection

Identifier

ComputeWorkloadAnalysis (Compute Workload Analysis)

InstructionStats (Instruction Statistics)

LaunchStats (Launch Statistics)

MemoryWorkloadAnalysis (Memory Workload Analysis)

NUMA Affinity (NumaAffinity)

Nvlink (Nvlink)

Nvlink_Tables (Nvlink_Tables)

Nvlink_Topology (Nvlink_Topology)

PM Sampling (PmSampling)

PM Sampling: Warp States (PmSampling_WarpStates)

SchedulerStats (Scheduler Statistics)

SourceCounters (Source Counters)

SpeedOfLight (GPU Speed Of Light Throughput)

WarpStateStats (Warp State Statistics)

Nsight Compute Profiling Tool (ncu)

Metric Collection: Default

- Default collection:
 - High-level utilization
 - Static Launch
 - Occupancy

Identifier

ComputeWorkloadAnalysis (Compute Workload Analysis)

LaunchStats (Launch Statistics)

MemoryWorkloadAnalysis (Memory Workload Analysis)

SpeedOfLight (GPU Speed Of Light Throughput)

WarpStateStats (Warp State Statistics)

Occupancy (Occupancy)

Nsight Compute Profiling Tool (ncu)

Example 1: vector-add-v2.py (N = 100,000)

- **Basic collection:**
 - `ncu python 3-vector-add-v0.py`

```
cliciad@gcn30 basic_examples]$ ncu -o output.txt python 1-vector-add-v0.py
```

```
==PROF== Connected to process 466370 (/gpfs/admin/_hpc/sw/arch/INTEL-  
AVX512/RHEL9/EB_production/2024/software/Python/3.12.3-GCCcore-13.3.0/bin/python3.12)
```

```
==PROF== Profiling "addition" - 0: 0%....50%....100% - 10 passes
```

```
Elapsed on GPU with PyCuda (sec): 1.3678835449218751
```

```
-----  
Elapsed time using CPU sequential for-loop (sec): 4.398094415664673
```

```
-----  
Validation: there are 0 different element(s)!
```

```
-----  
==PROF== Disconnected from process 1858485
```

```
[466370] python3.13@127.0.0.1
```

```
addition (19532, 1, 1)x(512, 1, 1), Context 1, Stream 7, Device 0, CC 8.0
```

Nsight Compute Profiling Tool (ncu)

Example 1: vector-add-v2.py (N = 100,000)

- **Basic collection:**
 - `ncu python 3-vector-add-v0.py`

```
cliciad@gcn30 basic_examples]$ ncu -o output.txt python 1-vector-add-v0.py
```

```
==PROF== Connected to process 466370 (/gpfs/admin/_hpc/sw/arch/INTEL-  
AVX512/RHEL9/EB_production/2024/software/Python/3.12.3-GCCcore-13.3.0/bin/python3.12)
```

```
==PROF== Profiling "addition" - 0: 0%....50%....100% - 10 passes
```

```
Elapsed on GPU with PyCuda (sec): 1.3678835449218751
```

```
-----  
Elapsed time using CPU sequential for-loop (sec): 4.398094415664673
```

```
-----  
Validation: there are 0 different element(s)!
```

```
-----  
==PROF== Disconnected from process 1858485
```

```
[466370] python3.13@127.0.0.1
```

```
addition (19532, 1, 1)x(512, 1, 1), Context 1, Stream 7, Device 0, CC 8.0
```

Nsight Compute Profiling Tool (ncu)

Example 1: vector-add-v2.py (N = 100,000)

Section: GPU Speed Of Light Throughput		
Metric Name	Metric Unit	Metric Value
DRAM Frequency	Ghz	1.20
SM Frequency	Ghz	1.08
Elapsed Cycles	cycle	4,428
Memory Throughput	%	12.76
DRAM Throughput	%	12.76
Duration	us	4.10
L1/TEX Cache Throughput	%	7.81
L2 Cache Throughput	%	14.60
SM Active Cycles	cycle	2,111.67
Compute (SM) Throughput	%	3.24

Section: GPU and Memory Workload Distribution		
Metric Name	Metric Unit	Metric Value
Average DRAM Active Cycles	cycle	627.10
Total DRAM Elapsed Cycles	cycle	198,656
Average L1 Active Cycles	cycle	2,111.91
Total L1 Elapsed Cycles	cycle	499,844
Average L2 Active Cycles	cycle	1,814.34
Total L2 Elapsed Cycles	cycle	344,160
Average SM Active Cycles	cycle	2,111.91
Total SM Elapsed Cycles	cycle	499,844
Average SMSP Active Cycles	cycle	2,204.97
Total SMSP Elapsed Cycles	cycle	1,999,376

Section: Launch Statistics		
Metric Name	Metric Unit	Metric Value
Block Size		512
Function Cache Configuration	CachePreference	None
Grid Size		196
Registers Per Thread	register/thread	16
Shared Memory Configuration Size	Kbyte	16.38
Driver Shared Memory Per Block	Kbyte/block	1.02
Dynamic Shared Memory Per Block	byte/block	0
Static Shared Memory Per Block	byte/block	0
# SMs	SM	108
Stack Size		1,024
Threads	thread	100,352
# TPCs		54
Enabled TPC IDs		all
Uses Green Context		0
Waves Per SM		0.45

Section: Occupancy		
Metric Name	Metric Unit	Metric Value
Block Limit SM	block	32
Block Limit Registers	block	8
Block Limit Shared Mem	block	16
Block Limit Warps	block	4
Theoretical Active Warps per SM	warp	64
Theoretical Occupancy	%	100
Achieved Occupancy	%	42.42
Achieved Active Warps Per SM	warp	27.15

OPT Est. Local Speedup: 57.58%
The difference between calculated theoretical (100.0%) and measured achieved occupancy (42.4%) can be the result of warp scheduling overheads or workload imbalances during the kernel execution. Load imbalances can occur between warps within a block as well as across blocks of the same kernel. See the CUDA Best Practices Guide (<https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html#occupancy>) for more details on optimizing occupancy.

OPT If you execute `__syncthreads()` to synchronize the threads of a block, it is recommended to have at least two blocks per multiprocessor (compared to the currently executed 1.8 blocks) This way, blocks that aren't waiting for `__syncthreads()` can keep the hardware busy.

OPT This kernel grid is too small to fill the available resources on this device, resulting in only 0.5 full waves across all SMs. Look at Launch Statistics for more details.

Nsight Compute Profiling Tool (ncu)

Example 1: vector-add-v2.py (N = 100,000)

```
OPT  Est. Speedup: 5.41%
      One or more SMs have a much lower number of active cycles than the average number of active cycles. Maximum
      instance value is 11.86% above the average, while the minimum instance value is 27.51% below the average.
-----
OPT  Est. Speedup: 6.235%
      One or more SMSPs have a much lower number of active cycles than the average number of active cycles. Maximum
      instance value is 13.09% above the average, while the minimum instance value is 28.62% below the average.
-----
OPT  Est. Speedup: 5.41%
      One or more L1 Slices have a much lower number of active cycles than the average number of active cycles.
      Maximum instance value is 11.86% above the average, while the minimum instance value is 27.51% below the
      average.
-----
OPT  Est. Speedup: 12.21%
      One or more L2 Slices have a much higher number of active cycles than the average number of active cycles.
      Maximum instance value is 28.96% above the average, while the minimum instance value is 9.33% below the
      average.
```

Nsight Compute Profiling Tool (ncu)

Example 1: vector-add-v2.py (N = 100,000,000)

Section: GPU Speed Of Light Throughput

Metric Name	Metric Unit	Metric Value
DRAM Frequency	Ghz	1.21
SM Frequency	Ghz	1.09
Elapsed Cycles	cycle	951,181
Memory Throughput	%	88.28
DRAM Throughput	%	88.28
Duration	us	868.70
L1/TEX Cache Throughput	%	16.20
L2 Cache Throughput	%	69.27
SM Active Cycles	cycle	944,526.70
Compute (SM) Throughput	%	12.21

Section: Occupancy

Metric Name	Metric Unit	Metric Value
Block Limit SM	block	32
Block Limit Registers	block	8
Block Limit Shared Mem	block	16
Block Limit Warps	block	4
Theoretical Active Warps per SM	warp	64
Theoretical Occupancy	%	100
Achieved Occupancy	%	85.51
Achieved Active Warps Per SM	warp	54.73

Section: Launch Statistics

Metric Name	Metric Unit	Metric Value
Block Size		512
Function Cache Configuration	CachePreferNone	
Grid Size		195,313
Registers Per Thread	register/thread	16
Shared Memory Configuration Size	Kbyte	16.38
Driver Shared Memory Per Block	Kbyte/block	1.02
Dynamic Shared Memory Per Block	byte/block	0
Static Shared Memory Per Block	byte/block	0
# SMs	SM	108
Stack Size		1,024
Threads	thread	100,000,256
# TPCs		54
Enabled TPC IDs		all
Uses Green Context		0
Waves Per SM		452.11

OPT Est. Local Speedup: 14.49%

The difference between calculated theoretical (100.0%) and measured achieved occupancy (85.5%) can be the result of warp scheduling overheads or workload imbalances during the kernel execution. Load imbalances can occur between warps within a block as well as across blocks of the same kernel. See the CUDA Best Practices Guide (<https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html#occupancy>) for more details on optimizing occupancy.

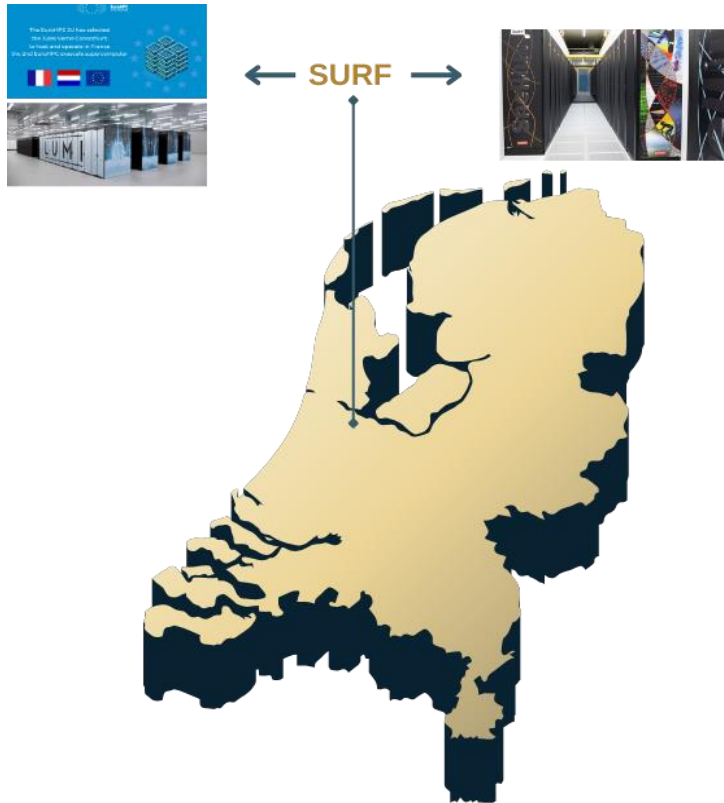
INF This workload is utilizing greater than 80.0% of the available compute or memory performance of the device. To further improve performance, work will likely need to be shifted from the most utilized to another unit. Start by analyzing DRAM in the Memory Workload Analysis section.

Thank You

Funded by the European Union. This work has received funding from the European High Performance Computing Joint Undertaking (JU) and Germany, Bulgaria, Austria, Croatia, Cyprus, Czech Republic, Denmark, Estonia, Finland, Greece, Hungary, Ireland, Italy, Lithuania, Latvia, Poland, Portugal, Romania, Slovenia, Spain, Sweden, France, Netherlands, Belgium, Luxembourg, Slovakia, Norway, Türkiye, Republic of North Macedonia, Iceland, Montenegro, Serbia under grant agreement No 101101903.

Access to Supercomputers and GPUs

Current



Future

- Alice Recoque Exascale System (France - 2026)
The Alice Recoque supercomputer will be hosted and operated by the Jules Verne consortium, led by France through GENCI and CEA, with the participation of the Netherlands through SURF.
- AI Factory Groningen - 2027
consortium partners SURF, TNO and Samenwerking Noord, has taken the initiative for this facility and is working on its implementation, aiming to have it fully operational by 2027.

Supercomputers

EuroHPC Systems



✓ PROCURED

5 PETASCALE

- Vega in Slovenia
- Karolina in Czechia
- Discoverer in Bulgaria
- Meluxina in Luxembourg
- Deucalion in Portugal

3 PRE-EXASCALE

- Lumi in Finland
- Leonardo in Italy
- Marenostrum 5 in Spain

⌚ ONGOING

2 EXASCALE

- Jupiter in Germany
- Alice Recoque in France

2 MID-RANGE

- Arrhenius in Sweden
- Daedalus in Greece

▶▶ COMING NEXT

UPGRADES

- Discoverer+
- Lisa/Leonardo

AN INDUSTRIAL SYSTEM

- Co-owned and for use by the industrial sector
- For AI and other applications

A POST-EXASCALE SYSTEM

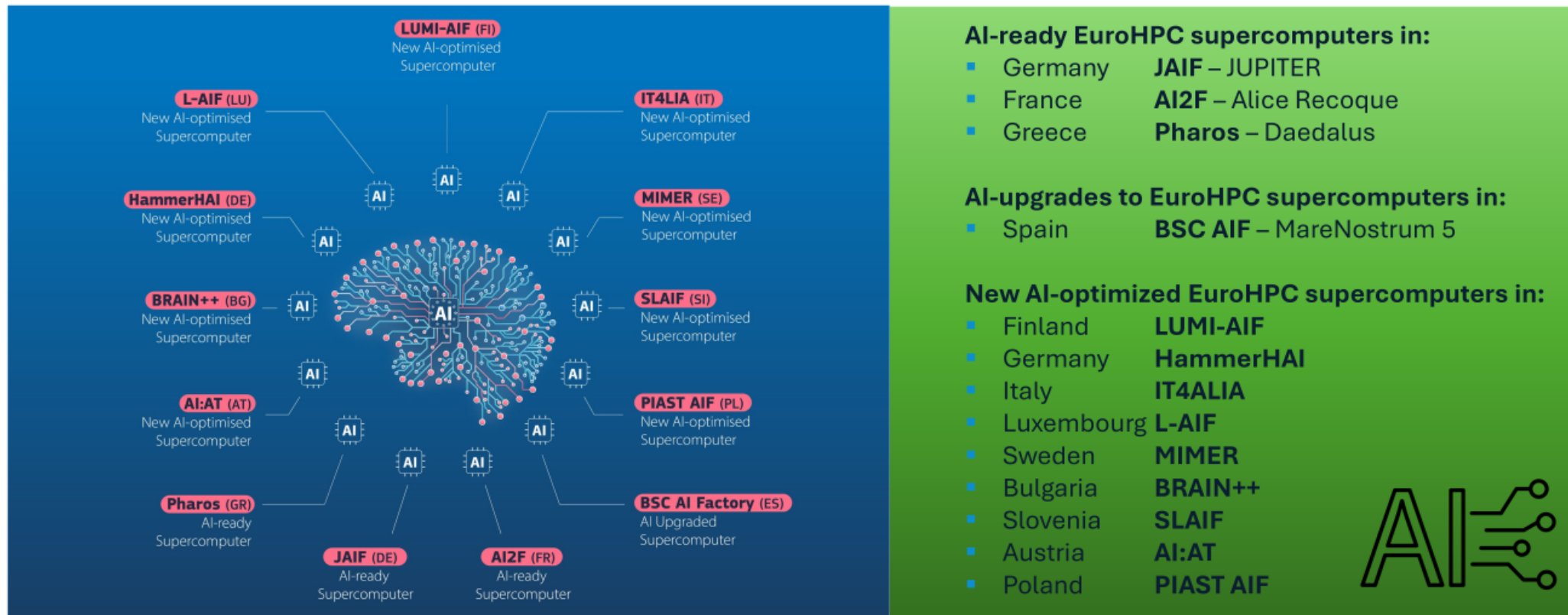
PROCUREMENT OF FEDERATION SERVICES

- A platform for the federation of EuroHPC HPC and quantum infrastructure
- A one-stop shop access point for users

Supercomputers AI factories (EuroHPC Systems)

THE AI FACTORIES

EuroHPC JU selected 13 EU sites that will host the first AI Factories – to drive Europe's leadership in AI



AI Factories pull together EU and national resources, in a collaborative effort of **21 European countries**